

Cyber Offence Simulations: Practical Analysis of Offensive Security Techniques

Abstract

The report provides a comprehensive examination of all offensive cybersecurity methods which Immersive Labs activities and their dedicated virtual penetration testing environment have demonstrated. The report investigates multiple cyber-attack incidents which involve OSINT and SQL Injection and Cross Site Scripting (XSS), and infrastructural exploitation and persistence methods and contemporary cyber threat patterns, web application attacks and IoT device vulnerabilities. The dedicated virtual testing environment was established with Kali Linux and Damn Vulnerable Web Application (DVWA) to create a testing setup which simulated SQL injection attacks and Stored XSS attacks. The document presents the attack methods together with their security risks and their connection to established frameworks, which include the MITRE ATT&CK framework and OWASP.

Table of Contents

Introduction.....	3
2 Immersive Labs Analysis.....	4
2.1 Introduction to Penetration Testing.....	4
2.2 OSINT.....	5
2.3 SQL Injection Basics.....	7
2.4 Cross-Site Scripting.....	8
2.5 Infrastructure Hacking.....	10
2.6 Emerging Threats.....	12
2.7 Introduction to Web Application Hacking.....	14
2.8 Persistence.....	16
2.9 IoT and Embedded Devices.....	17
3 Design and Deployment of a Virtual Testbed.....	18
3.1 SQL Injection Attack.....	19
3.2 Cross-Site Scripting Attack.....	19
Conclusion.....	23
References.....	24
Appendix.....	25

Introduction

Therefore, cybersecurity has become a vital issue for organisations due to the expansion of digital infrastructure and the increasing complexities and occurrences of cyber-attacks.

Today's organisations heavily depend on interconnected systems, web technologies, and cloud computing for day-to-day activities. Although these technologies offer organisations greater convenience and scalability, they are accompanied by a multitude of risks that can be leveraged by attackers for malicious purposes. In this regard, offensive security methodologies have become a vital part of designing effective security defences.

Offensive security aims at discovering vulnerabilities in organisations' systems before attackers can exploit them for malicious purposes. The best-known offensive security methodology is penetration testing and red teaming. These methodologies aim at emulating real-world cyber-attacks in a virtual environment for discovering security vulnerabilities, assessing defensive capabilities, and enhancing the security of organisations' systems. By using the perspective of attackers, security professionals can achieve deeper insights into security vulnerabilities and ways of addressing them.

This report provides an analysis of various simulations carried out on cyber offence activities through practical laboratory exercises. These activities include various attack scenarios such as Open-Source Intelligence Gathering, web application attacks including SQL injection and Cross-Site Scripting, infrastructure attacks, persistence, and new cyber threats. Through these activities, we aim to understand how attackers target vulnerabilities in computer systems and how such vulnerabilities may lead to security breaches.

In addition to this analysis, this report also aims to provide an overview of how an isolated virtual test environment was designed and implemented with an attacker machine and a target system vulnerable to cyber-attacks. Within this test environment, two different cyber-attacks are simulated with the aim of illustrating how offensive attack scenarios are carried out in real-world situations. An analysis is provided on each attack scenario based on its goals, tools used, methodologies, and results. This analysis is further discussed regarding various cybersecurity best practices and frameworks, illustrating how an organisation can be better protected from similar cyber-attacks.

The aim and objective of this report is not only to provide an illustration of how various cyber-attacks are carried out but also to critically analyse how effectively offensive methodologies can be implemented to provide better insights and recommendations on how an organisation can be better protected from various cyber-attacks and threats.

2 Immersive Labs Analysis

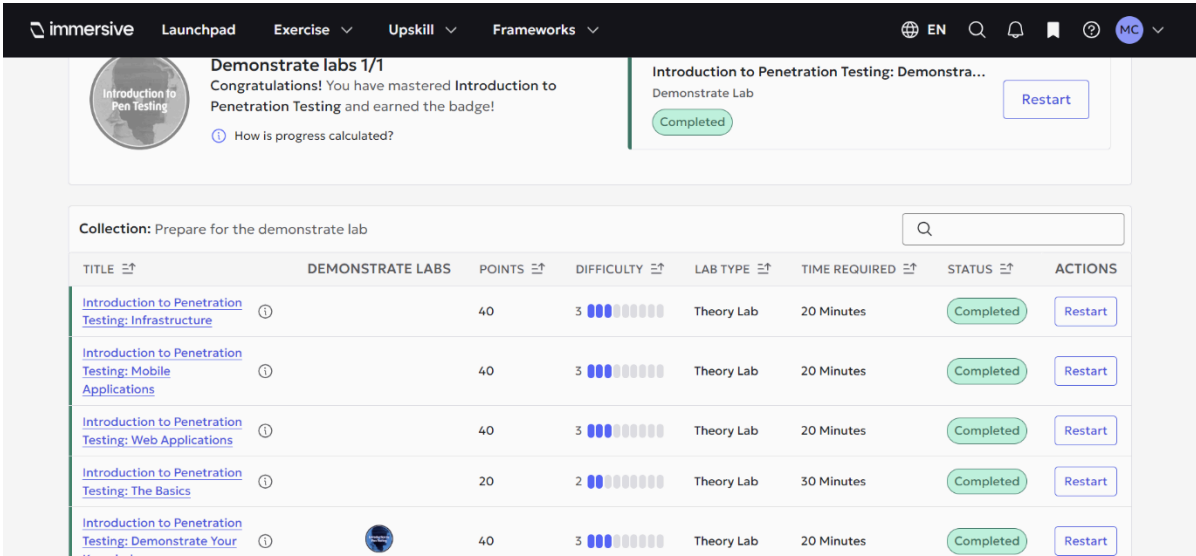
2.1 Introduction to Penetration Testing

Introduction to Penetration Testing Laboratories has offered a general overview of methodologies related to penetration testing and the areas that are evaluated as part of such security engagements. This set of labs considered basic elements of infrastructure, mobile application, and web application penetration testing, along with explaining essential concepts that are leveraged by security specialists during vulnerability assessments and simulated cyber-attacks.

In turn, the Infrastructure laboratory described how the process of network analysis, service exposure detection, and the use of insecure system configurations are used by attackers in order to gain unauthorised access to systems. As for the Mobile Applications lab, it outlined the security issues related to mobile devices and applications, namely, insecure data storage and poor authentication measures. Furthermore, the Web Applications laboratory highlighted typical vulnerabilities found in publicly available applications that become primary targets in most cyber-attacks.

Moreover, the Basics lab has outlined basic steps of penetration testing and the way these tests are executed, including the processes of reconnaissance, enumeration, exploitation, privilege escalation, and reporting. Therefore, these laboratory exercises have shown how penetration testing goes beyond the exploitation phase and includes analysis of how systems, their users, and configurations interact within a particular environment.

Finally, since the presented activities were theoretical laboratories, the key approach involved thorough analysis of the given briefs and learning material, followed by independent research in case some of the concepts needed additional elaboration from my side. When new terms and attack techniques came up, they were studied separately. To sum up, this set of theoretical laboratories has been quite helpful when preparing myself for practical penetration testing exercises.



The screenshot displays the Immersive Labs dashboard. At the top, a navigation bar includes 'immersive', 'Launchpad', 'Exercise', 'Upskill', and 'Frameworks'. On the right, there are icons for language (EN), search, notifications, and a user profile (MC). The main content area features a 'Demonstrate labs 1/1' section with a congratulatory message and a 'Restart' button. Below this is a table titled 'Collection: Prepare for the demonstrate lab' with columns for Title, Demonstrate Labs, Points, Difficulty, Lab Type, Time Required, Status, and Actions. The table lists five labs, all of which are marked as 'Completed'.

TITLE	DEMONSTRATE LABS	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS
Introduction to Penetration Testing: Infrastructure	①	40	3	Theory Lab	20 Minutes	Completed	Restart
Introduction to Penetration Testing: Mobile Applications	①	40	3	Theory Lab	20 Minutes	Completed	Restart
Introduction to Penetration Testing: Web Applications	①	40	3	Theory Lab	20 Minutes	Completed	Restart
Introduction to Penetration Testing: The Basics	①	20	2	Theory Lab	30 Minutes	Completed	Restart
Introduction to Penetration Testing: Demonstrate Your Knowledge	①	40	3	Theory Lab	20 Minutes	Completed	Restart

Figure 1 Introduction to Penetration Testing - Completed

2.2 OSINT

During the completion of the Open-Source Intelligence (OSINT) labs, learners could see how publicly available information could be acquired and analysed to support reconnaissance operations. The exercises showed that attackers could easily get significant amounts of intelligence about an organisation without even contacting the target systems directly, to reduce their chances of getting detected while carrying out the first stage of the attack.

In this respect, the Search Engines, Cached and Archived Websites, and Shodan.io labs helped understand how search engines and databases of internet-connected devices can be utilised to uncover potential vulnerabilities, hidden content, and exposed systems. The Robots.txt lab proved how configuration files of websites may help attackers discover directories and administrative areas that could be used during the reconnaissance process.

The labs focused on the aspects of online anonymity, investigator OPSEC, and privacy, also showing how critical it is for attackers and investigators to protect themselves from being traced and exposed. The Online Anonymity, Investigator Operations Security (OPSEC), and Tor labs helped understand the reasons why investigators need to ensure their privacy and anonymity and how they could achieve this aim.

Finally, it is possible to say that the EXIF and Boarding Pass labs helped see that file metadata can provide attackers with important data such as username, location, travel plans, timestamps, and hardware characteristics. Likewise, the Social Media, Deleted Tweet, and Social Media and Privacy labs showed that the contents of tweets and other posts might be exploited by malicious users to conduct targeted social engineering or phishing operations.

Finally, the Default Credentials lab demonstrated how using default passwords and usernames can make a company become a target of cyber criminals since all this data can be found in special public online databases. Thus, to prevent any attack, organisations should change default login credentials to complex and difficult-to-guess ones.

Considering the theory-based nature of most of the labs mentioned above, learners could apply a rather passive approach based on a close review of the study materials and online investigation to learn more about each concept. Overall, the OSINT labs have revealed how critically important information security practices and policies can reduce the amount of intelligence gathered about an organisation by attackers.

immersive

Launchpad

Exercise

Upskill

Frameworks

EN

MC



Completed

OSINT Badge

Congratulations you have mastered OSINT and earned the badge!

Collection

TITLE	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS
Open Source Intelligence (OSINT): Search Engines	20	2	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Online Anonymity	40	3	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Cached and Archived Websites	20	2	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Social Media and Privacy	10	1	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Investigator Operations Security (OPSEC)	40	3	Theory Lab	10 Minutes	Completed	Restart

Figure 2 OSINT - Completed

immersive

Launchpad

Exercise

Upskill

Frameworks

EN

MC

Open Source Intelligence (OSINT): Online Anonymity	40	3	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Cached and Archived Websites	20	2	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Social Media and Privacy	10	1	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Investigator Operations Security (OPSEC)	40	3	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Robots.txt	40	3	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): EXIF	40	3	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Social Media	40	3	Theory Lab	15 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Boarding Pass	100	4	Theory Lab	13 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Shodan.io	20	2	Theory Lab	10 Minutes	Completed	Restart
Tor	40	3	Theory Lab	10 Minutes	Completed	Restart
Open Source Intelligence (OSINT): Deleted Tweet	40	3	Theory Lab	13 Minutes	Completed	Restart

Figure 3 OSINT - Completed

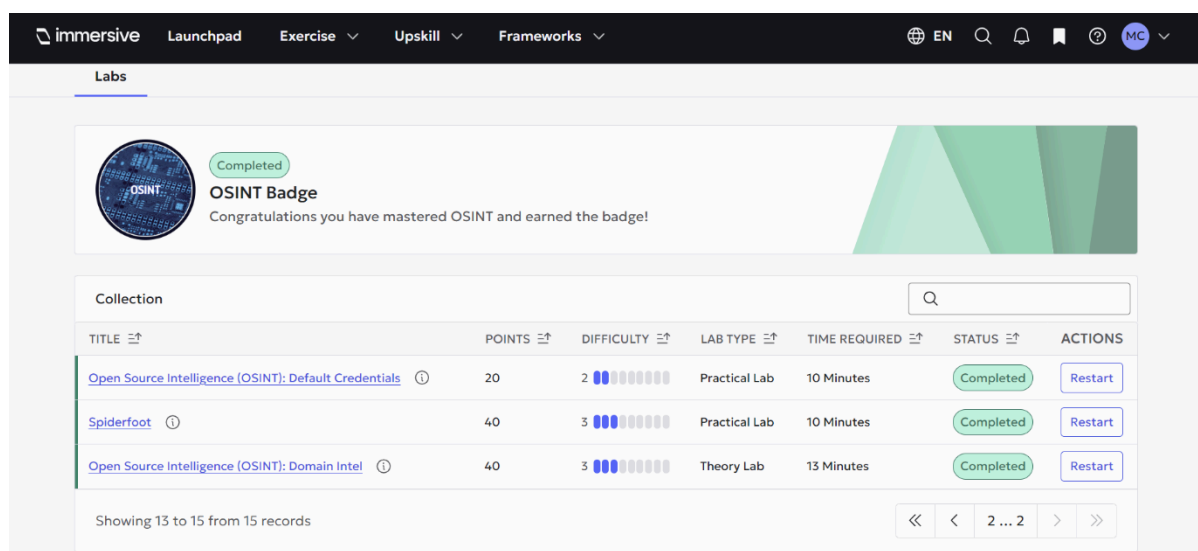


Figure 4 OSINT - Completed

2.3 SQL Injection Basics

The SQL Injection Basics laboratories offered valuable hands-on experience in detecting and exploiting SQL injection flaws in web-based applications. The successful completion of the exercises illustrated how hackers use their skills to manipulate user inputs to influence database queries and retrieve important data. All the labs showed potential threats to an application when database interaction is performed incorrectly or when insufficient input validation is implemented by the developer.

In the Basic SQL Injection lab and the Hidden Data lab, hackers were shown the methods to use URL parameters and other types of application inputs to get unauthorised access to information that should not normally be available. In these exercises, it became obvious how vulnerable a database could become to SQL injection attacks because of the absence of appropriate user input validation.

The UNION Query Lab and the Enumerating the Database lab taught how hackers use different SQL statements in combination with an application query to retrieve information from additional database tables. With these exercises, the learners acquired more knowledge about database structure enumeration, database table discovery, and data extraction methods.

Filter Evasion and String Concatenation exercises helped the participants understand how hackers use alternative query structures to bypass security filters. During such labs, it became obvious that SQL Injection attacks cannot be prevented by using simple blacklist filtering because attackers can modify the query in many ways to evade security measures.

As one part of the Demonstrating Your Skills project, a comprehensive test included various SQL Injection techniques in one scenario. As a result, learners had to implement several actions like reconnaissance, query manipulation, enumeration, filter evasion, etc. For a while, I had problems constructing effective queries and finding out why certain payloads did not work. However, all the SQL injection exercises increased my knowledge on how to identify and exploit such a vulnerability.

From a defensive point of view, the SQL injection labs gave me valuable information on the usage of parameterised queries, prepared statements, server-side user input validation, and least privilege accounts.

TITLE	DEMONSTRATE LABS	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS
SQL Basics: Basic SQL Injection	①	100	4	Practical Lab	10 Minutes	Completed	Restart
SQL Basics: UNION Query	①	200	5	Practical Lab	15 Minutes	Completed	Restart
SQL Basics: Hidden Data	①	100	4	Practical Lab	10 Minutes	Completed	Restart
SQL Basics: Filter Evasion	①	200	5	Practical Lab	13 Minutes	Completed	Restart
SQL Basics: String Concatenation	①	300	6	Practical Lab	36 Minutes	Completed	Restart
SQL Basics: Enumerating the Database	①	300	6	Practical Lab	27 Minutes	Completed	Restart
SQL Basics: Demonstrate Your Skills	①	400	7	Practical Lab	30 Minutes	Completed	Restart

Figure 5 SQL Injection Basics - Completed

2.4 Cross-Site Scripting

These laboratory sessions offered valuable hands-on learning in discovering different types of Cross-Site Scripting (XSS) flaws and executing exploits for their abuse. In particular, the labs explained how attackers may inject a malicious JavaScript payload into a trusted website to perform operations in the context of the victim's browser. Some of the possible effects of these attacks include stealing user cookies, manipulating webpage content, hijacking user sessions, and redirecting users to malicious websites.

In the first theoretical exercise, learners were introduced to Cross-Site Scripting and explained why this web application vulnerability continues to be widespread despite the availability of effective prevention techniques. As demonstrated in the subsequent labs on Reflected XSS and Stored XSS, attackers can inject payloads either in the response reflected by the server to the request or store them in the database, so the script runs each time the user loads an affected page. The danger of a stored XSS attack is that a malicious script may affect all users accessing a compromised page.

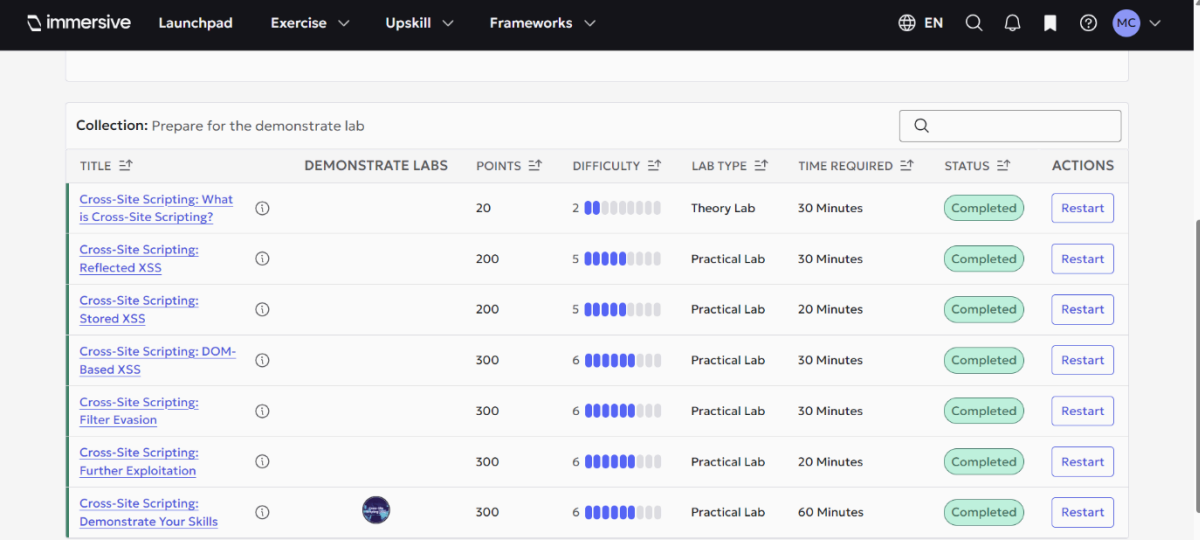
The DOM-Based XSS lab was aimed at showing students how some vulnerabilities may arise from the client-side JavaScript code only and thus cannot be prevented with server-side measures. The activity consisted of the need to detect and exploit a vulnerability in the Whisper Messaging search feature and perform an XSS attack by injecting a payload that would execute JavaScript code in the client's browser.

Filter Evasion and Further Exploitation lab activities showed how attackers may bypass insufficient input filters by modifying the payload syntax and implementing alternative means of running the JavaScript payload. The exercises demonstrated that blacklist filtering does not offer reliable protection against attacks since attackers have numerous ways to modify

payloads and bypass filters. In addition, labs also revealed how XSS could be combined with other attack techniques to escalate the effects of exploitation.

Demonstrate Your Skills assignment required students to combine several XSS exploits in a more complex scenario. This was one of the trickiest tasks since one had to bypass filter mechanisms while launching an attack on a stored XSS vulnerability that allowed to run injected JavaScript. Additionally, there were difficulties in finding out the exact location of the DOM-based vulnerability in the search section of the application. However, after making numerous modifications in payloads, both exploits were successfully performed, and JavaScript alerts appeared on the webpage.

In terms of defensive techniques, the XSS laboratory sessions revealed the necessity to practice safe output encoding, implement proper input validation, enable a Content Security Policy (CSP), and secure session management. Overall, this set of exercises proved very informative as they allowed me to better understand the potential risks related to improper handling of user input.



Collection: Prepare for the demonstrate lab							
TITLE	DEMONSTRATE LABS	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS
Cross-Site Scripting: What is Cross-Site Scripting?	①	20	2	Theory Lab	30 Minutes	Completed	Restart
Cross-Site Scripting: Reflected XSS	①	200	5	Practical Lab	30 Minutes	Completed	Restart
Cross-Site Scripting: Stored XSS	①	200	5	Practical Lab	20 Minutes	Completed	Restart
Cross-Site Scripting: DOM-Based XSS	①	300	6	Practical Lab	30 Minutes	Completed	Restart
Cross-Site Scripting: Filter Evasion	①	300	6	Practical Lab	30 Minutes	Completed	Restart
Cross-Site Scripting: Further Exploitation	①	300	6	Practical Lab	20 Minutes	Completed	Restart
Cross-Site Scripting: Demonstrate Your Skills	①	300	6	Practical Lab	60 Minutes	Completed	Restart

Figure 6 Cross-Site Scripting - Completed

2.5 Infrastructure Hacking

The attackers used insecure network services and weak authentication mechanisms together with system misconfigurations to demonstrate their method of accessing protected systems and confidential data. The activities that took place during this period matched actual penetration testing and red team testing because attackers used enumeration together with exposed services and weak credentials to conduct their attacks instead of using advanced malware.

The first exercise required participants to conduct an analysis of LDAP authentication traffic through Wireshark. The `ldap.bindRequest` filter enabled users to extract authentication requests from the packet capture file. The packet analysis showed that two users, Daveyjones and their password `IWantAllTheLoot789`, transmitted their usernames through the network link. The test showed that LDAP authentication ran without encryption, which led to credential information being intercepted through packet sniffing.

The security implications of this weakness are significant. Attackers who monitor network traffic can collect valid credentials, which they will use to gain unauthorised access and move through the network while increasing their access rights. The issue exists because the authentication traffic was not protected using encryption protocols such as LDAPS or TLS. Organisations should implement encrypted authentication methods and create separate zones for sensitive information, and they need to track network activity to detect credential theft attempts.

The latest infrastructure attack targeted FTP system enumeration and subsequent exploitation activities. Nmap scan results obtained through the command `nmap -sC -sV` showed an FTP service operating on port 2121, which permitted anonymous users to access the system. The anonymous FTP client connection led to the discovery and download of the `token.txt` file. The case showed that weak service configuration, together with faulty access control mechanisms, made it possible for unauthorised users to reach confidential documents.

Legacy systems and embedded systems still experience the same security problem because their administrators fail to properly configure anonymous FTP access on their systems. Actual attackers would use this security hole to steal confidential information while infecting systems with dangerous files and launching fresh attacks against the corporation. Organisations should disable anonymous access while they need to establish authentication protocols and implement file access limits, and they must replace their unsafe FTP system with secure SFTP encryption.

The laboratories provided SSH access using actual user credentials. Remote access to a Linux system was established through `sshpass` after the user entered their credentials, and then they used the command to copy the file `token.txt` from their home directory. The activity required users to enter lab-specific credentials, which demonstrates how attackers obtain remote access by using stolen or weak system credentials.

The attackers conducted an advanced infrastructure assault through Kerberoasting, which functioned within their Active Directory framework. The script `GetUserSPNs.py` from Impacket was used to track Service Principal Names (SPNs), which led to the discovery of the `svc-http` service account. The account received a Kerberos ticket, which generated a hash that hackers obtained through offline cracking to discover the password `Passw0rd!`. The attackers used these credentials to control the Domain Controller through remote

command execution, which allowed them to steal a secret token from the Administrator workstation.

The process of Kerberoasting creates deadly security risks because attackers can obtain service tickets without needing special access rights. The weakness arises when service accounts use weak or predictable passwords that can be cracked offline. An attacker exploited this vulnerability in enterprise environments to obtain domain access and achieve complete administrative rights. The organisation protects its systems through several defensive measures, which include creating strong passwords for service accounts, using managed service accounts, limiting SPN visibility, tracking unusual Kerberos ticket requests, and enforcing users to have only essential access rights.

The security labs demonstrate that attackers succeed through two main ways, which include exploiting system configuration errors and using insecure passwords and unsafe network protocols, and their network protections are not enough to prevent attacks. The organisation needs to adopt secure configuration management together with full data encryption and constant system monitoring, and authentication security measures to protect its entire infrastructure.

immersive Launchpad Exercise Upskill Frameworks EN Q ? MC									
TITLE	DEMONSTRATE LABS	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS		
Infrastructure Hacking: Bash History	①	100	4	Practical Lab	10 Minutes	Completed	Restart		
Infrastructure Hacking: XSL Script Processing	①	200	5	Practical Lab	25 Minutes	Completed	Restart		
Infrastructure Hacking: Sudo Caching	①	100	4	Practical Lab	20 Minutes	Completed	Restart		
Infrastructure Hacking: Using SearchSploit	①	200	5	Practical Lab	26 Minutes	Completed	Restart		
Infrastructure Hacking: SSL Scanning	①	200	5	Practical Lab	13 Minutes	Completed	Restart		
Infrastructure Hacking: SimpleHTTPServer	①	100	4	Practical Lab	15 Minutes	Completed	Restart		
Infrastructure Hacking: Space After Filename	①	100	4	Practical Lab	10 Minutes	Completed	Restart		
CVSS Calculator	①	300	6	Theory Lab	26 Minutes	Completed	Restart		
Infrastructure Hacking: XWD Screen Capture	①	200	5	Practical Lab	31 Minutes	Completed	Restart		
Infrastructure Hacking: OpenLDAP Plaintext	①	100	4	Practical Lab	14 Minutes	Completed	Restart		

Figure 7 Infrastructure Hacking - 12 labs completed

Infrastructure Hacking: FTP Anonymous Login	①	100	4	Practical Lab	10 Minutes	Completed	Restart
---	---	-----	---	---------------	------------	-----------	---------

Figure 8 Infrastructure Hacking - 12 labs completed

Infrastructure Hacking: Kerberoasting	①	200	5	Practical Lab	45 Minutes	Completed	Restart
---	---	-----	---	---------------	------------	-----------	---------

Figure 9 Infrastructure Hacking - 12 labs completed

2.6 Emerging Threats

The labs provided by Emerging Threats shed some light on how cyber-attacks have changed over time and what kind of new techniques hackers are using to bypass security mechanisms and perform attacks stealthily. As of now, hackers do not use only malware or exploitation to hack into digital environments because their strategies become more advanced, sophisticated, and involve automation, legitimate functionality of applications and systems, and other factors. By working on completed labs, it is possible to see the changes in hacking techniques and understand why defending against modern cyber-attacks becomes challenging for organisations.

The lab that analysed the CVE-2024-3094 xz Utils supply chain compromise provided useful insights into this type of attack. It showed that in some cases, threat actors use their access to an organisation or its software supply chain to inject malicious code into a software package. Such attacks reflect the rising popularity of supply chain compromise that allows hackers to deliver malware through a series of trusted components and avoid detection by anti-malware software. The exercises demonstrated that modern cyber-attacks increasingly require the use of more elaborate techniques, allowing for the simultaneous infection of multiple systems.

In particular, the lab about the malicious Python package highlighted this issue. In this case, threat actors upload malicious packages into popular package repositories for developers to download. In this way, the latter unknowingly install packages that contain malicious code and expose themselves to the risk of becoming part of a botnet. Such attacks are extremely dangerous since they allow adversaries to target hundreds or thousands of organisations simultaneously simply by uploading malicious packages into commonly used repositories.

Furthermore, the Lazarus cyber espionage campaign lab offered a chance to see how modern advanced persistent threat (APT) actors conduct cyber operations against their targets. In this exercise, the learner had to analyse a campaign in which attackers deployed a combination of phishing emails, malware, social engineering, and other techniques to obtain access to systems. Similarly, the Storm-0978 threat actor investigation lab showed what kind of actions organised cybercriminal groups can perform, namely, exploiting vulnerabilities and engaging in social engineering activities aimed at delivering ransomware.

Moreover, the labs allowed looking closer at modern threat vectors like deepfakes, malicious remote monitoring and management (RMM) tools, and fileless persistence mechanisms. The first lab demonstrated the use of modern artificial intelligence technology for impersonating someone else to perform a variety of activities, such as spreading a ransomware strain across corporate networks. On the other hand, the Atera and MeshCentral labs demonstrated the use of modern malicious RMM tools designed to control victims' computers in a stealth manner. The software in question cannot be detected easily since it does not look like any suspicious program.

On balance, from a security standpoint, the Emerging Threats labs show the necessity to implement a proactive approach based on several security solutions. First, it is imperative for security analysts to implement behavioural analytics and correlate threat data to detect malicious activities. Second, software integrity verification is essential to prevent the use of malicious applications or supply chain infections. Third, an endpoint detection and response solution is highly recommended to identify suspicious activities. Finally, employees should undergo regular cybersecurity training.

immersive Launchpad Exercise Upskill Frameworks							
TITLE	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS	
OpenClaw: Investigating Malicious Skills ⓘ	100	4	Practical Lab	30 Minutes	Completed	Restart	
Investigating a ClickFix Intrusion ⓘ	200	5	Practical Lab	60 Minutes	In Progress	Continue	
APT37: Ruby Jumper Campaign – Analysis ⓘ	400	7	Practical Lab	40 Minutes	In Progress	Continue	
7-Zip Installer (Proxy Node Campaign): Analysis ⓘ	300	6	Practical Lab	30 Minutes	Completed	Restart	
Lotus Blossom: Notepad++ Campaign Analysis ⓘ	300	6	Practical Lab	30 Minutes	Completed	Restart	
Shai-Hulud 2.0: Analysis ⓘ	300	6	Practical Lab	30 Minutes	Completed	Restart	
MalTerminal: Malware Analysis ⓘ	200	5	Practical Lab	30 Minutes	Completed	Restart	
Scattered Spider and DragonForce: Campaign Analysis ⓘ	300	6	Practical Lab	30 Minutes	In Progress	Continue	
Norwegian Dam Compromise: Campaign Analysis ⓘ	300	6	Practical Lab	30 Minutes	Completed	Restart	
Zero-Day Behavior: PDF Samples ⓘ	200	5	Practical Lab	60 Minutes	Completed	Restart	

Figure 10 Emerging Threats - 17/20 labs completed

Zero-Day Behavior: PDF Samples ⓘ	200	5	Practical Lab	60 Minutes	Completed	Restart
Threat Actors: Storm-0978 ⓘ	100	4	Practical Lab	30 Minutes	Completed	Restart
Threat Hunting: Analyzing Malicious Python Packages ⓘ	200	5	Practical Lab	60 Minutes	Completed	Restart

Figure 11 Emerging Threats - 17/20 labs completed

immersive Launchpad Exercise Upskill Frameworks							
TITLE	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS	
Malware: Analyzing Temporary .NET Assemblies ⓘ	300	6	Practical Lab	30 Minutes	In Progress	Continue	
Deepfakes ⓘ	40	3	Practical Lab	20 Minutes	Completed	Restart	
CVE-2024-3094 (xz) – Supply Chain Compromise ⓘ	300	6	Practical Lab	45 Minutes	Completed	Restart	
GhostEngine: Analysis ⓘ	200	5	Practical Lab	30 Minutes	Completed	Restart	
TypeLib: COM Persistence ⓘ	200	5	Practical Lab	30 Minutes	Completed	Restart	
Water Gamayun: (CVE-2025-26633) Campaign Analysis ⓘ	300	6	Practical Lab	30 Minutes	Completed	Restart	
BadSuccessor: Defensive ⓘ	200	5	Practical Lab	60 Minutes	Not started	Start	
BadSuccessor: Offensive ⓘ	200	5	Practical Lab	60 Minutes	Not started	Start	
Detecting RMM Tools: Atera ⓘ	300	6	Practical Lab	30 Minutes	Completed	Restart	
Detecting RMM Tools: MeshCentral ⓘ	300	6	Practical Lab	30 Minutes	Completed	Restart	
Lazarus Cyber Espionage Campaign: Analysis ⓘ	300	6	Practical Lab	30 Minutes	Completed	Restart	

Figure 12 Emerging Threats - 17/20 labs completed

2.7 Introduction to Web Application Hacking

Introduction to Web Application Hacking labs have offered a good practical experience on identifying vulnerabilities that exist in modern web applications. The activities have shown how hackers are able to perform reconnaissance, analyse application behaviour, craft HTTP requests, and use insecure application configuration to penetrate a computer system.

Mapping Web Applications and Dirbuster labs have shown how attackers try to discover the directories, files, and administrative pages that are hidden from users and not available via web applications. In particular, the custom headers activity has helped to demonstrate how HTTP request headers can be manipulated by attackers. Such exercises have proven the significance of conducting reconnaissance before starting a penetration test and revealed that exposing resources will result in an increased attack surface.

Page Source Review and Stack Trace labs have shown that attackers can obtain sensitive information from the source code and error messages of the application under investigation. Source code analysis has helped to reveal hidden functionality and internal references. Meanwhile, the stack trace has provided technical details about the technologies used in the web application. Thus, the importance of secure error handling has been proven.

The activities like Authentication Design Flaws, Apache Basic Authentication, and Bypassing HTTP Client-Side Controls have revealed that client-side authentication cannot ensure the security of the application. The experiments have shown that hackers can easily intercept and modify client-side requests.

The labs related to the Server Side Includes, Command Injection, and Directory Traversal have illustrated that attackers might be able to use insecure input validation to launch an attack on the target web application. As a result, the system may be seriously compromised.

Finally, the Introduction to OWASP ZAP lab has helped to improve understanding of automated testing for vulnerabilities. The tool helped to identify weaknesses in the applications under test by intercepting their traffic and performing analyses. Initially, I had difficulties with interpreting responses correctly and configuring scans. However, such activities have improved my skills in web application testing and penetration testing.

In conclusion, the labs on web application hacking have shown that secure coding and testing are critical in minimising the number of potential vulnerabilities of the web application.

Completed

Introduction to Web App Hacking Badge

Congratulations you have mastered Introduction to Web App Hacking and earned the badge!

Collection

TITLE	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS
Intro to Web App Hacking: Mapping Web Applications ⓘ	200	5	Practical Lab	23 Minutes	Completed	Restart
Intro to Web App Hacking: Dirbuster – Introduction ⓘ	200	5	Practical Lab	17 Minutes	Completed	Restart
Intro to Web App Hacking: Dirbuster – Custom Headers ⓘ	200	5	Practical Lab	45 Minutes	Completed	Restart
Intro to Web App Hacking: Page Source Review ⓘ	200	5	Practical Lab	10 Minutes	Completed	Continue
Intro to Web App Hacking: Introduction to OWASP ZAP ⓘ	300	6	Practical Lab	60 Minutes	Completed	Restart

Figure 13 Introduction to Web Application Hacking - Completed

TITLE	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS
Intro to Web App Hacking: Dirbuster – Introduction ⓘ	200	5	Practical Lab	17 Minutes	Completed	Restart
Intro to Web App Hacking: Dirbuster – Custom Headers ⓘ	200	5	Practical Lab	45 Minutes	Completed	Restart
Intro to Web App Hacking: Page Source Review ⓘ	200	5	Practical Lab	10 Minutes	Completed	Continue
Intro to Web App Hacking: Introduction to OWASP ZAP ⓘ	300	6	Practical Lab	60 Minutes	Completed	Restart
Intro to Web App Hacking: Bypassing HTTP Client-Side Controls ⓘ	200	5	Practical Lab	30 Minutes	Completed	Restart
Intro to Web App Hacking: Authentication Design Flaws ⓘ	200	5	Practical Lab	30 Minutes	Completed	Restart
Intro to Web App Hacking: Server Side Includes ⓘ	200	5	Practical Lab	12 Minutes	Completed	Restart
Intro to Web App Hacking: Command Injection ⓘ	100	4	Practical Lab	12 Minutes	Completed	Restart
Intro to Web App Hacking: Stack Trace ⓘ	100	4	Practical Lab	13 Minutes	Completed	Restart
Intro to Web App Hacking: Apache Basic Authentication ⓘ	200	5	Practical Lab	19 Minutes	Completed	Restart
Intro to Web App Hacking: Directory Traversal ⓘ	200	5	Practical Lab	10 Minutes	Completed	Restart

Figure 14 Introduction to Web Application Hacking – Completed

2.8 Persistence

The Persistence exercises revealed that hackers use persistent access techniques to retain access to their targets for long periods. Instead of using conventional malware, hackers exploit normal operating system functions and administrative tools to create backdoors that cannot be removed easily, even after rebooting systems. The labs conducted showed that hackers may use Windows registry manipulation, creation of Windows services, activation of accessibility features, scripting of startup activities, and manipulation of authentication mechanisms to gain and maintain access while avoiding detection.

Similarly, the Persistence: Trap lab showcased that even very basic persistence mechanisms may allow attackers to automatically run malware whenever certain users' activities happen. Also, the Persistence: .bashrc Files lab highlighted that Linux bash shell initialisation script files (.bashrc) may allow hackers to execute arbitrary code upon starting a shell session.

The Persistence: Accessibility Features and Incident Response: Persistence via Accessibility Features exercises revealed that malicious actors may leverage legitimate Windows features to achieve the execution of code without authenticating users first.

It is evident from this experiment that hackers often substitute valid system binaries with malicious files. In these experiments, it was demonstrated that hackers exploit legal Windows components to establish access and stay undetected; yet they require incident response analysis to detect persistence exploits and confirm system integrity.

Persistence: Component Object Model (COM) Hijacking and Persistence: Image File Execution Options (IFEO) Injection experiments indicated that the Windows Registry can be manipulated to deceive genuine applications into executing malicious files. In the IFEO Injection experiment, it was shown that the exploitation of persistence exploits allowed hackers to divert genuine application (.exe) execution towards malicious files. Even though non-malicious programs were executed in these experiments, attackers could exploit similar persistence exploits to launch legitimate applications on malware or keylogger utilities. Such exercises proved that attackers use normal Windows functions to hide their activities and evade detection and investigation.

The Persistence: Windows Logon exercise showed how malicious actors configure their malware to execute automatically whenever users undergo the process of authentication.

The Persistence: Windows Services lab was one of the most difficult exercises because it involved identifying malicious services by examining multiple running services. Specifically, the exercise involved locating a suspicious Windows service installed outside of the software installation window for Immersive Bank, where the lab exercise took place. After investigating suspicious activity in Immersive Bank, the file path C:\ProgramData\Microsoft OneDrive\WmiCoreSrv.exe was found as the origin of that activity. However, it was challenging to identify which specific service should be identified as malicious because Windows runs too many other background services. Overall, this lab improved understanding of how attackers manipulate Windows services to execute malicious code and retain persistence with elevated privileges. Additionally, the lab showed the importance of forensic utilities such as AutoRuns, Procmon, and Windows Event Viewer.

Defensively speaking, the persistent labs revealed the importance of Windows registry monitoring, EDRs, PowerShell logging, Windows service auditing, and other continuous

system monitoring practices. Organisations should limit administrative privileges, monitor unusual startups, and conduct periodic system reviews to detect persistence-related activities of hackers.

TITLE	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS
Tactics: Persistence ⓘ	20	2	Theory Lab	15 Minutes	Completed	Restart
Persistence: Trap ⓘ	100	4	Practical Lab	10 Minutes	Completed	Restart
Persistence: Component Object Model Hijacking ⓘ	200	5	Practical Lab	24 Minutes	Completed	Restart
Persistence: Windows Logon ⓘ	200	5	Practical Lab	30 Minutes	Completed	Restart
Persistence: bashrc Files ⓘ	200	5	Practical Lab	10 Minutes	Completed	Restart
Persistence: Accessibility Features ⓘ	200	5	Practical Lab	31 Minutes	Completed	Restart
Persistence: Image File Execution Options (IFEO) Injection ⓘ	200	5	Practical Lab	35 Minutes	Completed	Restart
Incident Response: Persistence via Accessibility Features ⓘ	200	5	Practical Lab	18 Minutes	Completed	Restart
Persistence: Windows Services ⓘ	300	6	Practical Lab	25 Minutes	In Progress	Continue

Figure 15 Persistence - 8/9 labs completed

2.9 IoT and Embedded Devices

The IoT and Embedded Devices labs provided an understanding of the security implications associated with Internet of Things devices and embedded systems. Different from the desktop systems, the IoT and embedded devices do not offer adequate security controls, outdated firmware versions, and insecure network protocols for securing sensitive data and preventing any unauthorised actions. Completing the labs showed that attackers exploit IoT embedded systems to access the target networks, steal information, and disrupt the environment.

In the beginning, the introductory labs provided details about the basic architecture of IoT systems, various types of protocols used for communication, and different security issues related to connected devices. The labs confirmed that many of the IoT systems lack robust encryption measures, use insecure default credentials, and do not offer mechanisms for applying updates. It is important to recognise that IoT creates an enormous attack surface in organisations and private homes because of the number of interconnected devices.

The Firmware Credentials and Binwalk labs allowed practising the extraction of the firmware image of embedded systems and analysing their content in terms of possible security issues. The Binwalk tool was used to analyse embedded images for any hidden data, configuration, and sensitive credentials. In other words, the attacker can reverse the firmware of the IoT device to obtain encryption keys, credentials, passwords, and other information related to the IoT system. The initial difficulties were encountered when analysing the extracted firmware image due to its large size.

The Certificate Underpinning lab emphasised the importance of implementing certificate validation and secure communication within embedded systems. In particular, the lab demonstrated how the insecure implementation of certificates can allow the attacker to

intercept the traffic between the IoT systems or perform other activities. The lab made it clear that secure and proper certificate management is vital for ensuring the security of IoT networks.

Finally, the Supply Chain Hardware Tampering lab focused on attacking IoT devices through malicious alteration during their manufacturing or distribution process. As shown, embedded hardware devices can be easily modified by an attacker to cause disruptions or create an additional risk associated with the presence of embedded hardware components within the device.

Regarding defence, the labs provided valuable insights regarding firmware management, authentication, communication, and hardware protection against malicious attacks. Organisations should avoid using default passwords, continuously monitor connected systems and configure embedded hardware systems securely to prevent IoT attacks.

 Immersive

Launchpad

Exercise

Upskill

Frameworks

 EN









 MC

Collection

Q

TITLE	POINTS	DIFFICULTY	LAB TYPE	TIME REQUIRED	STATUS	ACTIONS
IoT & Embedded Devices: What Is IoT?	10	1	Theory Lab	10 Minutes	Completed	Restart
IoT & Embedded Devices: Security Issues	10	1	Theory Lab	10 Minutes	Completed	Restart
IoT and Embedded Devices: Network Protocols and Security	10	1	Theory Lab	10 Minutes	Completed	Restart
IoT & Embedded Devices: Hardware Reverse Engineering	10	1	Theory Lab	10 Minutes	Completed	Restart
IoT & Embedded Devices: Best Practice	10	1	Theory Lab	10 Minutes	Completed	Restart
IoT & Embedded Devices: Firmware Credentials	300	6	Practical Lab	24 Minutes	Completed	Restart
IoT & Embedded Devices: Certificate Underpinning	600	8	Practical Lab	20 Minutes	Completed	Continue
IoT & Embedded Devices: Binwalk	200	5	Practical Lab	10 Minutes	Completed	Restart
IoT & Embedded Devices: Supply Chain Hardware Tampering	100	4	Practical Lab	17 Minutes	Completed	Restart

Figure 16 IoT and Embedded Devices - Completed

3 Design and Deployment of a Virtual Testbed

To create an isolated and minimal virtual penetration testing environment, Oracle VirtualBox was utilised. An isolated environment was created for the purpose of simulating offensive security techniques in a safe way. Virtualisation technology allows security testing of computer systems without influencing any other system, network, or the host operating system itself. It also allows the user to easily restore and monitor their activities.

For the purposes of this lab session, the Kali Linux operating system was chosen as the attacker machine. The chosen operating system is commonly used as a penetration testing operating system, which features several offensive security and network analysis tools out of the box. A pre-configured image of Kali Linux was downloaded and imported as a virtual machine. Once the image was imported, the operating system was launched and set up as a virtual machine. All penetration testing tasks were completed within the virtual machine.

As the vulnerable application, Damn Vulnerable Web Application (DVWA) was chosen. As mentioned before, this application was created in such a way as to provide multiple security weaknesses that could be used by students in order to exercise offensive security techniques. The application was downloaded and hosted locally within the virtual machine. This application was reached via internal address – <http://127.0.0.1/>. The database environment was set up within the application's configuration by choosing the "Create / Reset Database" option. The setup creates a vulnerable application that can be used in penetration testing tasks.

During this practice session, the exploitation of SQL Injection and Cross-Site Scripting (XSS) was chosen due to their previous exploration in Immerse Lab tasks and inclusion in the list of common web application security vulnerabilities (OWASP Top 10 and MITRE ATT&CK frameworks). Screenshots were taken during the process of the attack process, payload execution, successful exploitation, and extracted information.

3.1 SQL Injection Attack

In real-world environments, SQL Injection vulnerabilities can have severe organisational consequences, including data breaches, credential theft, and loss of customer trust (OWASP, 2024). The attack also demonstrates how a single vulnerable input field may allow attackers to compromise an entire backend database if secure development practices are not followed. The aim of this exploit was to show that poor processing of user input can be taken advantage of by attackers to inject malicious code and manipulate database queries to gain access to confidential information stored in the backend database. The SQL Injection vulnerability was found in the DVWA SQL Injection module, following setting the security level of the application to "Low".

Simple payloads like `1 OR 1=1` were injected into the vulnerable input field of the DVWA application to confirm that the database query processes user input directly. Following the execution of the above payload, there was an output of multiple user records, hence confirming that user inputs in the DVWA app were not properly sanitised. Once the existence of the vulnerability in the application had been confirmed, a payload based on union was executed to successfully retrieve users' username and password hashes stored in the DVWA database.

Exploitation of this vulnerability was possible due to poor coding practices by the developers, whereby they did not use parameterised statements or implement proper server-side validations. The SQL Injection exploit demonstrated in this exercise falls under the MITRE ATT&CK technique T1190 - Exploit Public-Facing Application. To mitigate SQL Injection attacks, organisations need to implement the following defensive mechanisms: prepared statements, parameterised queries, server-side input validations, and least privilege database permissions.

3.2 Cross-Site Scripting Attack

Another practical attack carried out in the test environment included the exploitation of a Stored Cross-Site Scripting (XSS) flaw in the DVWA application. The objective of the attack was to show how attackers can leverage their privileges and inject JavaScript code into a trusted web application to perform any action inside the session of their victims. To test the application security vulnerabilities, the DVWA XSS (Stored) module was used after changing the DVWA application security level to "Low".

During the exploit, a payload consisting of `<script>alert('XSS')</script>` was added in the vulnerable guestbook feature of the application. Once the payload was submitted, the application stored the script and automatically executed it when the web page was accessed. The appearance of the JavaScript alert message confirmed the absence of output sanitisation and encoding on the part of the web application when displaying the payload.

It can be concluded that the attack was successful since the application did not have any adequate output encoding and input validation features, which allowed malicious code execution inside a user's browser. The vulnerability under discussion is classified as MITRE ATT&CK technique T1059.007 – JavaScript. In order to prevent Cross-Site Scripting attacks in practice, organisations should consider implementing proper measures such as context-sensitive output encoding, input validation, CSP protection, and session management.

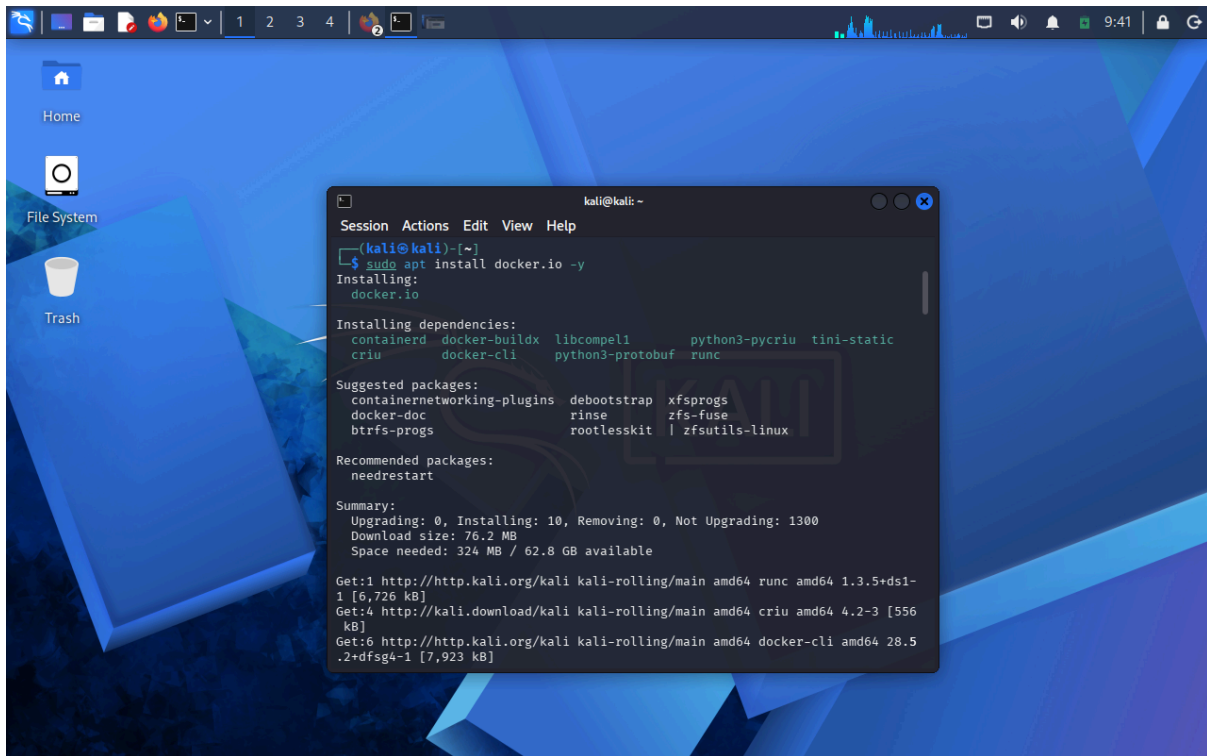


Figure 17 Docker Installation on Kali Linux Virtual Machine

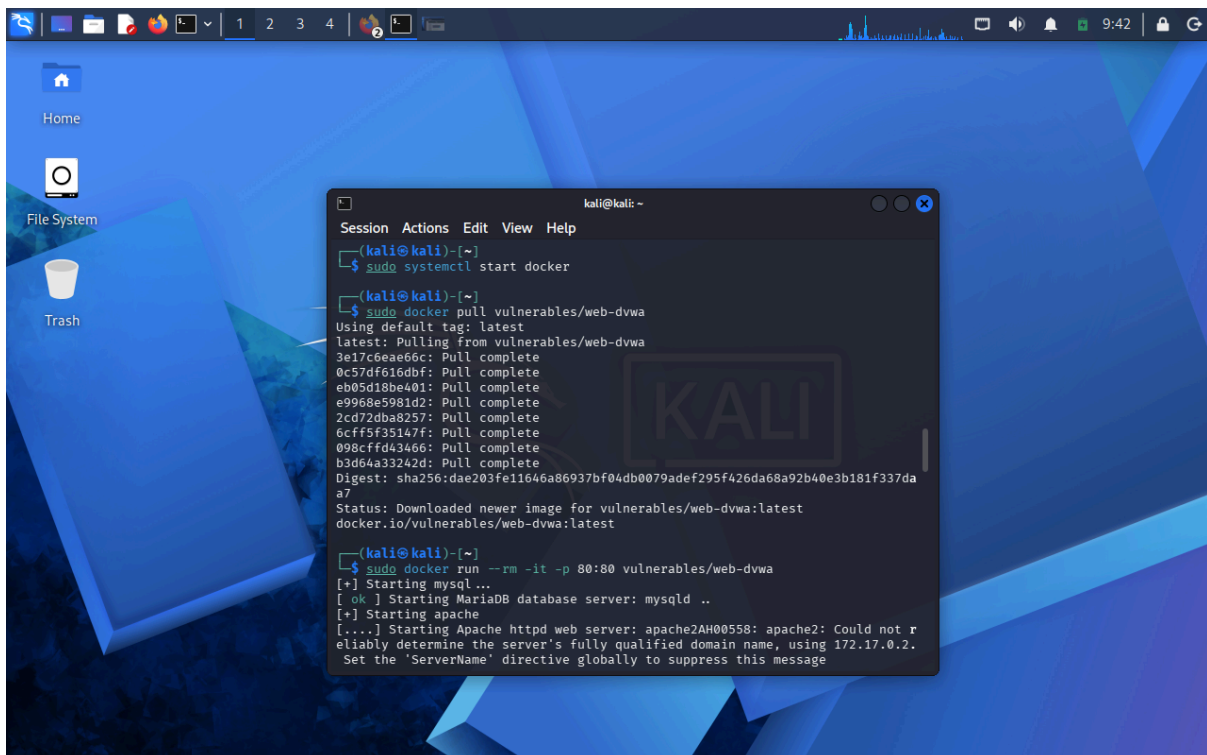


Figure 18 Running DVWA through Docker on Kali Linux

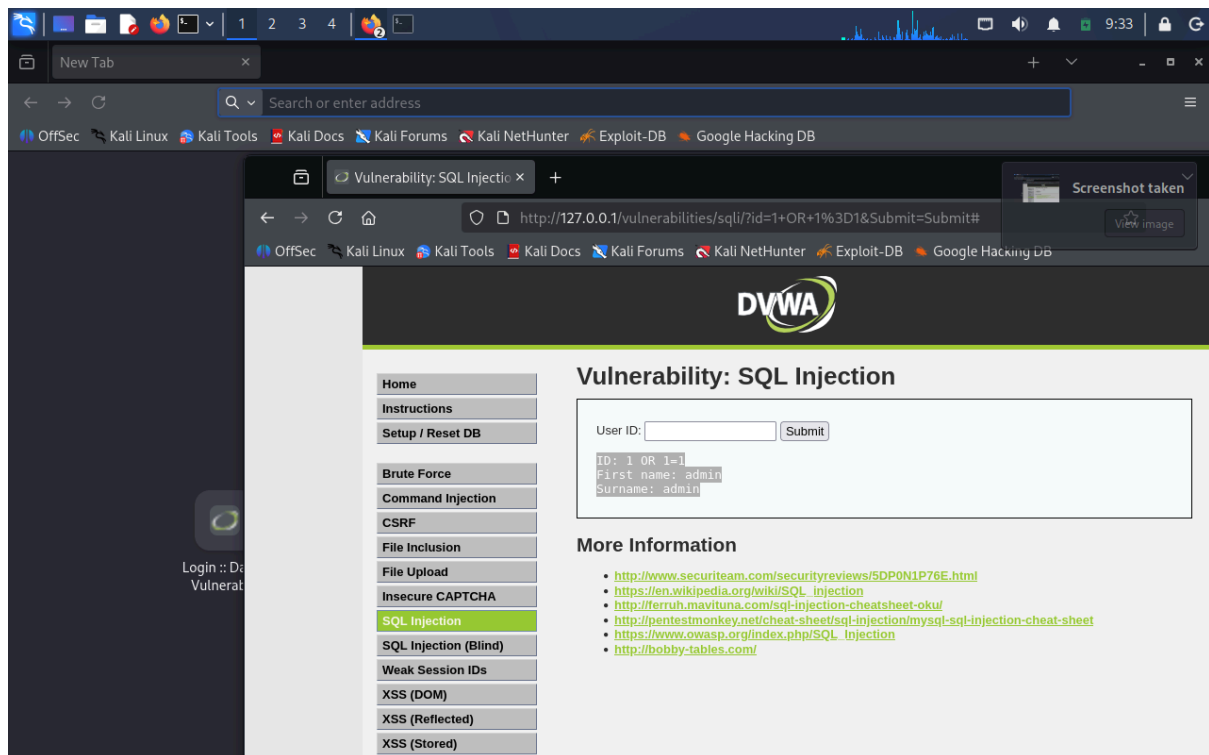


Figure 19 Successful Authentication Bypass using SQL Injection

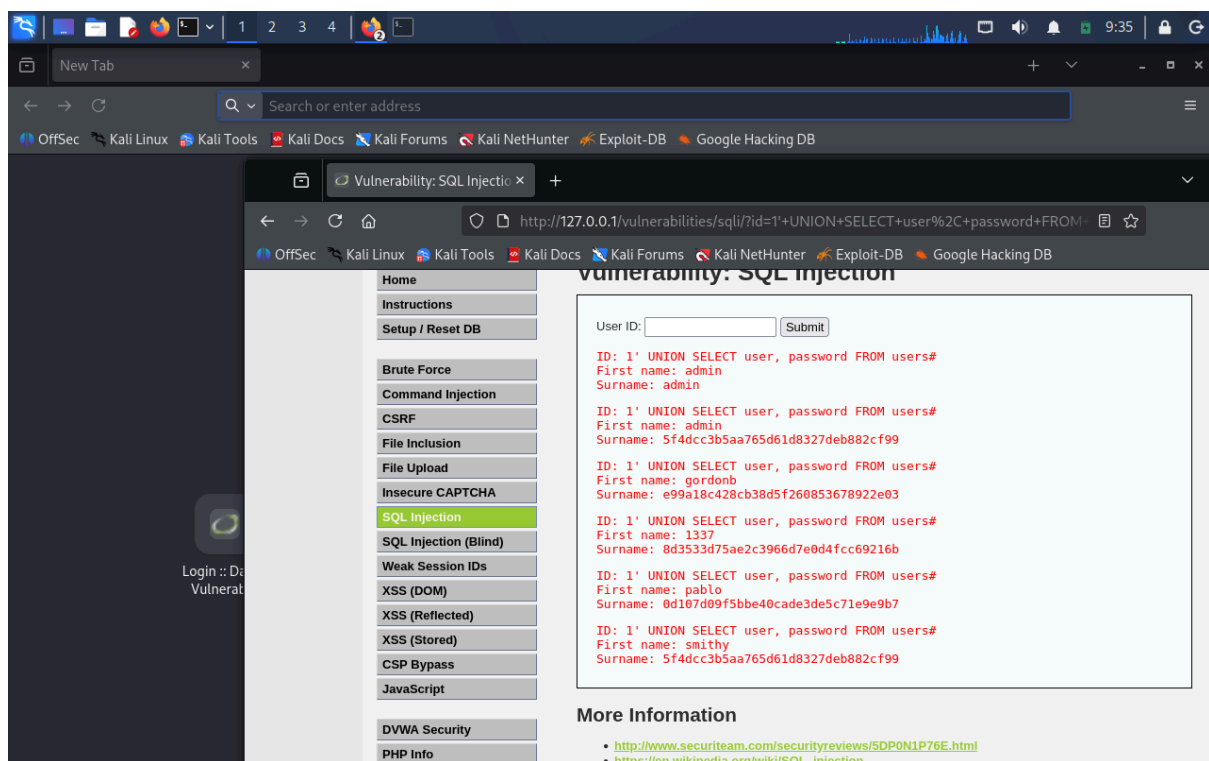


Figure 20 Extraction of User Credentials through UNION-Based SQL Injection

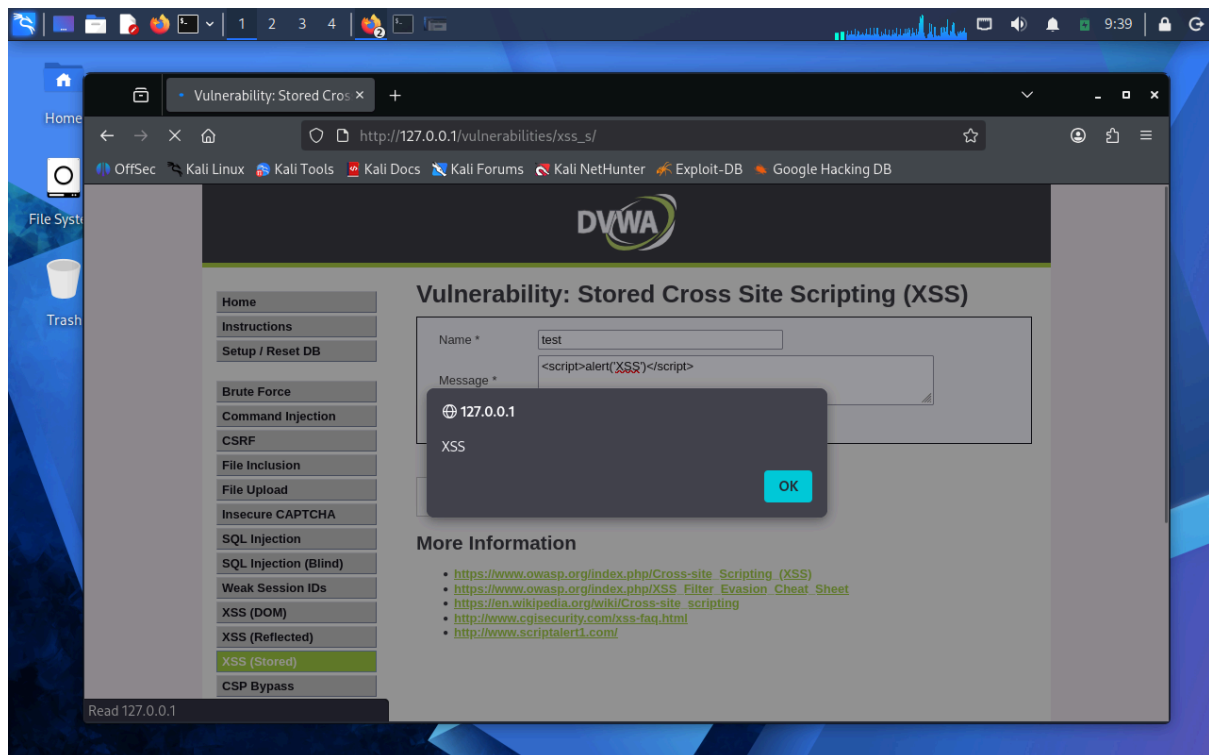


Figure 21 Stored XSS Payload Executed within DVWA

Conclusion

Overall, this assessment brought both theoretical and practical experience in multiple aspects of offensive security. More specifically, through completing the various Immersive Labs exercises described in Sections 2.1 to 2.9, insight into how attackers find and use vulnerabilities to exploit them for malicious purposes to persist and remain undetected can be gained.

The activities undertaken in Section 2.2 show how publicly available intelligence, metadata, caching, as well as poor operations security, can expose valuable data that can be used by the attackers. Furthermore, Sections 2.3 and 2.4 highlight the importance of creating a secure codebase when developing web applications since any potential vulnerability, such as the ability to use SQL injection, as well as cross-site scripting to launch a malicious script on the end-users' computers, should be prevented. The virtual lab environment created in Section 3 shows how easy it can be to execute both the SQL injection and stored cross-site scripting attacks on the DVWA website installed on the virtual machine.

The infrastructure-based exercises described in Section 2.3 show how poor authentication methods, service exposure, insecure protocols, and misconfiguration make an organisation susceptible to being attacked. Moreover, the persistence techniques shown in Section 2.8 demonstrate that attackers can leverage insecure configurations, weak passwords, as well as use such functionalities as accessibility features and COM objects hijacking for maintaining a persistent presence in the attacked system.

The exercises conducted in Section 2.5 describe the various emerging threats and malware-related analysis techniques, showing that the contemporary attacks usually include supply chain compromise, social engineering, malware, remote monitoring tools, as well as attacker behaviour. The exercises in Section 2.9 related to Internet of Things devices and their exploitation demonstrate how poorly written firmware, as well as using default settings when connecting these devices, lead to new attack vectors.

All in all, the above demonstrates that this assessment improved cybersecurity knowledge through providing both theoretical foundations for offensive and defensive strategies as well as practical exercises. The latter helped to understand various processes associated with the penetration test workflow, exploiting vulnerabilities, as well as preventing malicious activities.

Additionally, it was apparent from the evaluation that there is a need to maintain a balance between offensive cybersecurity measures and ethical considerations. While penetration tests and simulations are necessary in helping detect any vulnerabilities that might be exploited by cybercriminals, they should always be conducted within authorised environments. Through the practical sessions, it was evident that modern organisations need security assessments, constant monitoring, employee training, and secure coding practices to help address the cyber-attack threat. As cyber-attacks become increasingly complex, companies should implement multiple layers of security that will help counter the increasing threats.

References

GCHQ (2019). *CyberChef*. [online] Github.io. Available at: <https://gchq.github.io/CyberChef/>.

immersivelabs.online. (n.d.). *Immersive Labs*. [online] Available at: <https://immersivelabs.online/>.

Kali Linux. (n.d.). *Get Kali*. [online] Available at: <https://www.kali.org/get-kali/#kali-virtual-machines>.

MITRE (2025). *MITRE ATT&CK*. [online] Mitre.org. Available at: <https://attack.mitre.org/>.

Oracle (2023). *Oracle VM VirtualBox*. [online] VirtualBox. Available at: <https://www.virtualbox.org/>.

OWASP (2024). *SQL Injection*. [online] OWASP. Available at: https://owasp.org/www-community/attacks/SQL_Injection.

Wood, R. (2021). *diginiinja/DVWA*. [online] GitHub. Available at: <https://github.com/diginiinja/DVWA>.

